

Day 3 — Controller, Service, Repository Layers

Today you're taking yesterday's basic REST controller and organizing the application into **three layers**:

```
Client
  ↓
Controller
  ↓
Service
  ↓
Repository
```

The important thing is to understand **what each layer is responsible for**. We are intentionally using an **in-memory List instead of a database**.

1. Why Do We Need Layers?

Imagine putting everything into one controller:

```
@RestController
public class BookController {

    private List books = new ArrayList<>();

    @GetMapping("/books")
    public List getBooks() {
        return books;
    }

    @GetMapping("/books/{id}")
    public Book getBook(@PathVariable int id) {
        for (Book book : books) {
            if (book.id() == id) {
                return book;
            }
        }
    }
}
```

```
        return null;
    }
}
```

It works for a tiny application.

But as the application grows, the controller starts handling:

- HTTP requests
- business rules
- data storage
- searching
- validation
- database operations

That's not a good design.

Instead:

```
Controller
  ↓
Service
  ↓
Repository
```

Each layer has a specific responsibility.

2. Controller Layer

The **Controller** is the HTTP boundary of the application.

It deals with things such as:

- URLs
- HTTP methods
- path variables
- request parameters
- HTTP responses

Example:

```
@RestController
@RequestMapping("/books")
public class BookController {
}
```

The controller should generally **not** contain business logic or data-storage logic.

Simple definition

Controller receives HTTP requests, delegates the work, and returns HTTP responses.

3. Service Layer

The **Service** contains business/application logic.

For example:

```
If book exists → return it
If book doesn't exist → return empty result
```

Later, business rules might include:

```
Book price must be positive
Book title cannot be empty
Only available books can be borrowed
Maximum 5 books per user
```

Those rules belong primarily in the service layer.

Simple definition

Service contains the application's business logic and coordinates operations between the controller and repository.

4. Repository Layer

The **Repository** handles data access.

Today we're not using a database.

We'll simply use:

```
List
```

The repository will be responsible for storing and retrieving books from that list.

Simple definition

Repository provides an abstraction for accessing application data.

Today:

```
Repository
  ↓
In-memory List
```

Later:

```
Repository
  ↓
Database
```

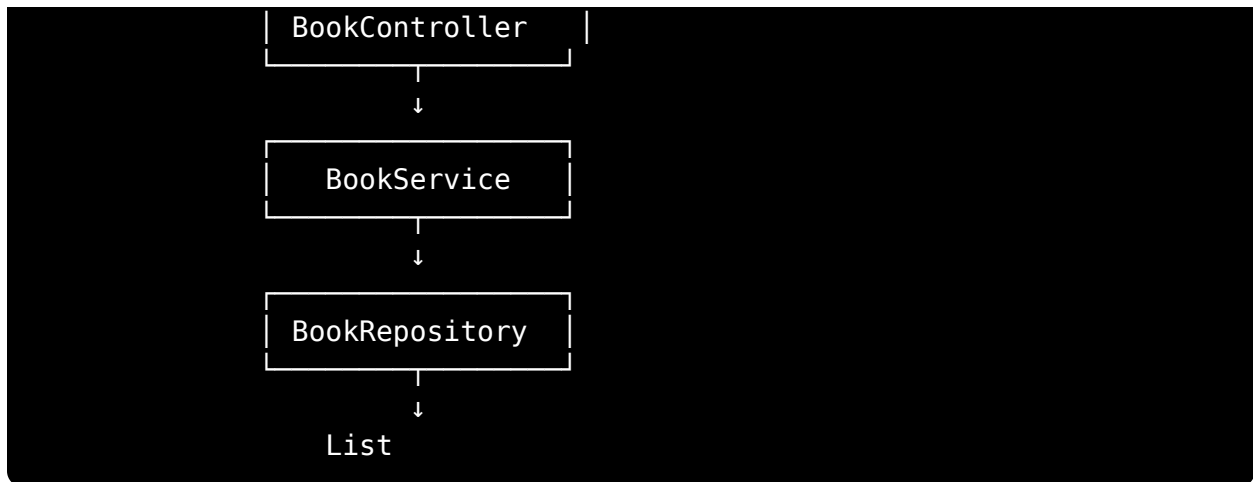
5. Complete Architecture

Our application will look like this:

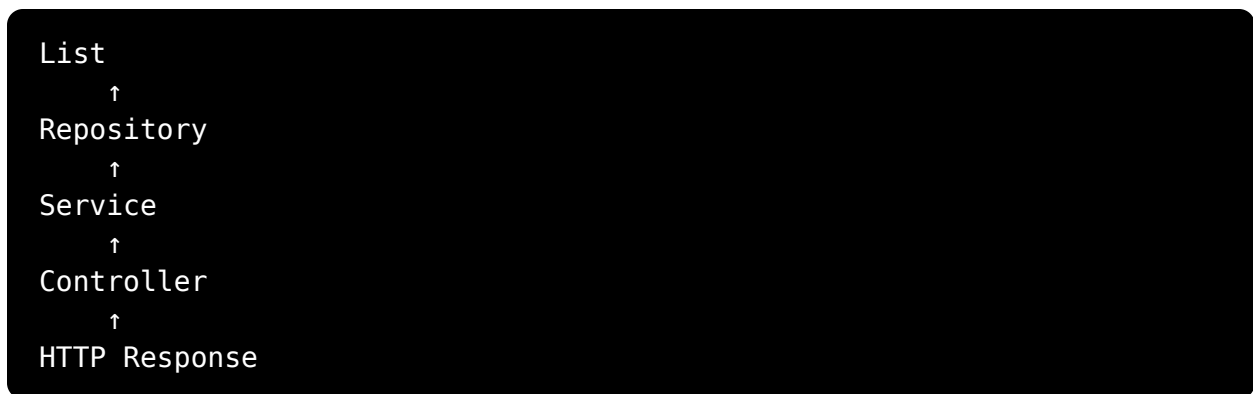
```
HTTP Request
```

```
↓
```

```
└──────────────────┘
```



And the response travels back:



6. Create the Book Model

We'll use a Java record because our model is simple.

```
package com.example.demo;

public record Book(
    int id,
    String title,
    String author
) {
}
```

A book might look like:

```
id: 1
title: Java Basics
author: James
```

7. Create BookRepository

The repository will contain our in-memory data.

```
package com.example.demo;

import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public class BookRepository {

    private final List books = List.of(
        new Book(1, "Java Basics", "James"),
        new Book(2, "Spring Boot", "Robert")
    );

    public List findAll() {
        return books;
    }

    public Optional findById(int id) {
        return books.stream()
            .filter(book -> book.id() == id)
            .findFirst();
    }
}
```

Notice:

```
@Repository
```

This tells Spring that this is a repository/data-access component.

8. Why Use Optional?

Our repository might find a book:

```
ID = 1  
  ↓  
Book found
```

Or not find one:

```
ID = 99  
  ↓  
No book
```

Instead of returning null, we use:

```
Optional
```

For example:

```
public Optional findById(int id)
```

It means:

"There may or may not be a Book."

At this stage, just understand the concept. You don't need advanced Optional knowledge yet.

9. Create BookService

Now the service needs the repository.

```
package com.example.demo;

import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public List getAllBooks() {
        return bookRepository.findAll();
    }

    public Optional getBookById(int id) {
        return bookRepository.findById(id);
    }
}
```

This is **constructor injection** from Day 2.

The service does not create:

```
new BookRepository()
```

Instead:

```
public BookService(BookRepository bookRepository)
```

Spring injects it.

10. Create BookController

Now the controller needs the service.

```
package com.example.demo;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/books")
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @GetMapping
    public List getBooks() {
        return bookService.getAllBooks();
    }

    @GetMapping("/{id}")
    public ResponseEntity getBookById(@PathVariable int id) {
        return bookService.getBookById(id)
            .map(ResponseEntity::ok)
            .orElseGet(() ->
                ResponseEntity.ok("Book not found"));
    }
}
```

Now we have the complete three-layer structure.

11. Complete Project Structure

```
src
├── main
│   └── java
│       └── com.example.demo
│           ├── DemoApplication.java
│           ├── Book.java
│           ├── BookController.java
│           ├── BookService.java
│           └── BookRepository.java
```

Conceptually:

```
BookController
    ↓
BookService
    ↓
BookRepository
    ↓
List
```

12. How GET /books Works

Client sends:

```
GET /books
```

Step 1 — Controller receives request

```
@GetMapping
public List getBooks() {
    return bookService.getAllBooks();
}
```

Controller says:

```
"I received a request for books. I'll ask the service."
```

Step 2 — Service receives request

```
public List getAllBooks() {  
    return bookRepository.findAll();  
}
```

Service says:

```
"I'll ask the repository for the data."
```

Step 3 — Repository gets data

```
public List findAll() {  
    return books;  
}
```

Repository returns:

```
[  
    Book(1, "Java Basics", "James"),  
    Book(2, "Spring Boot", "Robert")  
]
```

Step 4 — Response travels back

```
Repository  
  ↓  
Service  
  ↓
```

```
Controller
  ↓
Spring converts objects to JSON
  ↓
Client
```

Response:

```
[
  {
    "id": 1,
    "title": "Java Basics",
    "author": "James"
  },
  {
    "id": 2,
    "title": "Spring Boot",
    "author": "Robert"
  }
]
```

13. GET /books/{id} Flow

Suppose the client sends:

```
GET /books/1
```

The complete flow is:

```
Client
  |
  | GET /books/1
  ↓
BookController
  |
  | getBookById(1)
  ↓
```

```
BookService
|
|  getBookById(1)
↓
BookRepository
|
|  findById(1)
↓
List
|
|  Book found
↓
BookService
↓
BookController
↓
JSON Response
```

14. What Happens When ID Doesn't Exist?

Suppose:

```
GET /books/99
```

There is no book with ID 99.

Repository:

```
public Optional findById(int id) {
    return books.stream()
        .filter(book -> book.id() == id)
        .findFirst();
}
```

returns:

```
Optional.empty()
```

The service passes that result to the controller.

Our controller returns:

```
Book not found
```

So:

```
GET /books/99
```

produces:

```
Book not found
```

For this beginner exercise, that's fine.

In production APIs, you would typically return a proper HTTP 404 Not Found response. You'll learn better error-response design later.

15. Activity — Empty Result Message

The task says:

Return an empty result message when the book ID does not exist.

A simple implementation is:

```
@GetMapping("/{id}")
public ResponseEntity getBookById(@PathVariable int id) {
    return bookService.getBookById(id)
        .map(ResponseEntity::ok)
        .orElseGet(() ->
```

```
}
    ResponseEntity.ok("Book not found"));
}
```

Existing book

```
GET /books/1
```

Response:

```
{
  "id": 1,
  "title": "Java Basics",
  "author": "James"
}
```

Non-existing book

```
GET /books/99
```

Response:

```
Book not found
```

16. Why Not Put the List in Controller?

Bad organization

```
@RestController
public class BookController {

    private final List books = List.of(
        new Book(1, "Java Basics", "James"),
```

```
        new Book(2, "Spring Boot", "Robert")
    );

    @GetMapping("/books")
    public List getBooks() {
        return books;
    }
}
```

The controller now knows how data is stored.

That's unnecessary.

Better

```
Controller
  ↓
Service
  ↓
Repository
  ↓
List
```

The controller doesn't care whether the data comes from:

```
List
```

or later:

```
MySQL
```

or:

```
PostgreSQL
```

or:

MongoDB

That is one of the main benefits of layering.

17. What Does "Repository Abstraction" Mean?

Suppose the controller asks:

```
bookService.getAllBooks();
```

It doesn't need to know:

"Are books stored in a List?"

The service asks:

```
bookRepository.findAll();
```

The service also doesn't need to know the storage details.

Today:

```
BookRepository  
  ↓  
List
```

Later:

```
BookRepository  
  ↓  
JPA/Hibernate
```

↓
Database

The upper layers can remain mostly focused on their own responsibilities.

18. Dependency Injection Between Layers

We now have **two injections**.

Controller receives Service

```
private final BookService bookService;

public BookController(BookService bookService) {
    this.bookService = bookService;
}
```

Service receives Repository

```
private final BookRepository bookRepository;

public BookService(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
}
```

Spring creates the dependency graph:

```
BookRepository
  ↑
  |
BookService
  ↑
  |
BookController
```

Spring handles the object creation and wiring.

19. Why @Repository?

We have:

```
@Repository
public class BookRepository {
}
```

@Repository tells Spring that this class is a repository/data-access component.

It is also a specialization of @Component.

Similarly:

```
@Component
├─ @Service
└─ @Repository
```

These annotations help communicate the role of the class.

20. Layer Responsibilities

Layer	Main responsibility
Controller	HTTP/API handling
Service	Business/application logic
Repository	Data access
Model	Represents data

Controller

Deals with:

```
GET /books/1
@PathVariable
HTTP response
```

Service

Deals with:

```
Business rules
Coordination
Application logic
```

Repository

Deals with:

```
Finding data
Saving data
Deleting data
```

Model

Represents:

```
Book
```

21. What Should NOT Go Where?

Controller should not normally:

```
Directly access database
Contain complex business rules
Contain large data-processing logic
```

Service should not normally:

```
Handle HTTP-specific details
Know about URL paths
```

Repository should not normally:

```
Handle HTTP requests
Contain controller logic
```

Think:

```
HTTP → Controller
Business → Service
Data → Repository
```

22. Example of Business Logic

Suppose we introduce a simple rule:

Only books with a positive ID are valid.

The service could apply the rule:

```
public Optional getBookById(int id) {
    if (id <= 0) {
        return Optional.empty();
    }

    return bookRepository.findById(id);
}
```

The controller shouldn't have to decide whether `id <= 0` is a business rule.

The service handles it.

23. Why Keep Controller Small?

A good controller might look like:

```
@GetMapping
public List getBooks() {
    return bookService.getAllBooks();
}
```

That's simple.

The controller essentially says:

"I received this HTTP request; service, please handle it."

This makes the code easier to:

- understand
 - test
 - maintain
 - extend
-

24. Bad vs Good Architecture

Bad

```
Controller
├─ HTTP logic
├─ business logic
├─ data storage
├─ searching
├─ validation
└─ database operations
```

Good

```
Controller
└─ HTTP handling
    ↓
Service
└─ Business logic
    ↓
Repository
└─ Data access
    ↓
Data source
```

25. Practice — GET /books

Request:

```
GET http://localhost:8080/books
```

Expected:

```
[
  {
    "id": 1,
    "title": "Java Basics",
    "author": "James"
  },
  {
    "id": 2,
    "title": "Spring Boot",
    "author": "Robert"
  }
]
```

26. Practice — GET /books/1

Request:

```
GET http://localhost:8080/books/1
```

Expected:

```
{  
  "id": 1,  
  "title": "Java Basics",  
  "author": "James"  
}
```

27. Practice — GET /books/99

Request:

```
GET http://localhost:8080/books/99
```

Expected:

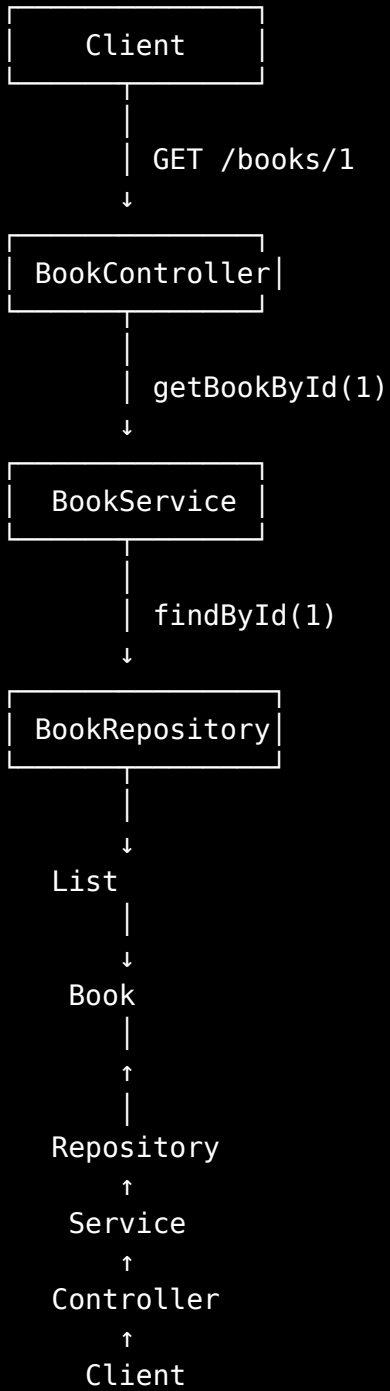
```
Book not found
```

28. The Request Flow You Should Be Able to Draw

For:

GET /books/1

draw:



This is an excellent diagram to understand for interviews.

29. Interview Questions

Q1. What are the three main layers in a basic Spring REST application?

Answer:

Controller, Service, and Repository.

`Controller → Service → Repository`

Q2. What is the responsibility of a Controller?

Answer:

The controller handles the HTTP/API boundary. It receives requests, extracts request data, delegates work to the service, and returns responses.

Q3. What is the responsibility of a Service?

Answer:

The service contains business/application logic and coordinates operations between controllers and repositories.

Q4. What is the responsibility of a Repository?

Answer:

The repository provides an abstraction for accessing and managing application data.

Q5. Why shouldn't the controller directly access the repository?

Answer:

It creates unnecessary coupling between the HTTP layer and data-access layer and makes the controller harder to maintain and test.

Q6. Why do we use a Service layer?

Answer:

It separates business logic from HTTP handling and keeps controllers thin.

Q7. What is the normal request flow?

Answer:

Client → Controller → Service → Repository → Data Source

The result travels back in the opposite direction.

Q8. Why is the repository using an in-memory list?

Answer:

For learning purposes, it allows us to understand the Controller-Service-Repository architecture without introducing database configuration and persistence complexity.

Q9. What happens if a book doesn't exist?

Answer:

The repository can return an empty Optional, the service passes that result upward, and the controller can return an appropriate response such as a "Book not found" message. In a production REST API, this would commonly be represented by HTTP 404.

Q10. Why use constructor injection between layers?

Answer:

It makes dependencies explicit, supports final fields, reduces coupling, and makes the classes easier to unit test.

30. Important Interview Question

Interviewer:

Explain the flow when a client requests GET /books/1.

Strong answer:

The request first reaches BookController because it has a GET /books/{id} mapping. The controller extracts the ID using @PathVariable and delegates the operation to BookService. The service contains the application logic and calls BookRepository to find the book. The repository accesses the in-memory list and returns the result. The result travels back through the service to the controller, and Spring converts the Book object into JSON for the HTTP response.

That's a very good beginner-level Spring answer.

31. Day 3 — What You Must Remember

```
Controller = HTTP
Service = Business Logic
Repository = Data Access
```

And:

```
GET /books/1
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
List
```

Core rules

1. **Controller receives HTTP requests.**
2. **Controller delegates work to Service.**
3. **Service contains business/application logic.**
4. **Service calls Repository.**

5. **Repository handles data access.**
6. Today's repository uses an **in-memory List**.
7. Don't put the hardcoded data inside the controller.
8. Don't put business logic directly inside the controller.
9. Use **constructor injection** between layers.
10. Think of the architecture as:

```
Client
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Data
```

The key idea is not the amount of code. It's understanding **why each class exists and why the request moves through these layers instead of putting everything inside the controller.**